微积分

求函数 $y = x^3e^{2x}$ 的导数。

解答

本题可根据乘积求导法则$(uv)' = u'v + uv'$（其中u、v为关于x的函数）来求解。

设$u = x^3$，$v = e^{2x}$。

1. **分别求u和v的导数**：
   - 根据求导公式$(x^n)' = nx^{n-1}$，对$u = x^3$求导，可得$u' = (x^3)' = 3x^2$。
   - 根据求导公式$(e^{ax})' = ae^{ax}$，对$v = e^{2x}$求导，可得$v' = (e^{2x})' = 2e^{2x}$。
2. **应用乘积求导法则求y的导数**：
   $y' = (x^3e^{2x})' = u'v + uv' = 3x^2e^{2x} + x^3 \times 2e^{2x} = 3x^2e^{2x} + 2x^3e^{2x}$ 。

综上，函数$y = x^3e^{2x}$的导数为$3x^2e^{2x} + 2x^3e^{2x}$。

：在函数 x^3，x^2+1，$2x+3$ 中，哪个函数是双射函数。

双射函数定义

双射函数（bijective function）也叫一一对应函数。它是一种特殊的函数，同时满足**单射**和**满射**的性质。

- **单射（injective）**：对于函数 $f:A\to B$，如果对于 A 中任意两个不同的元素 x_1 和 x_2，都有 $f(x_1)\neq f(x_2)$，即不同的自变量对应不同的函数值。
- **满射（surjective）**：对于函数 $f:A\to B$，B 中的每一个元素 y，在 A 中都至少存在一个元素 x 使得 $f(x)=y$，也就是函数的值域等于目标集合 B。

当一个函数既满足单射又满足满射时，就称它为双射函数。这意味着在定义域 A 和值域 B 的元素之间存在着一一对应的关系，即可以通过函数建立起 A 和 B 中元素的精确配对，并且可以从 B 到 A 进行唯一的逆操作。

- $y=x^3$ 是双射函数。从单射角度看，对于不同的 x_1 和 x_2，$x_1^3\neq x_2^3$；从满射角度看，对于任意实数 y，都能找到实数 $x=\sqrt[3]{y}$ 使得 $y=x^3$，它的定义域和值域都是实数集 R。
- $y=x^2+1$ 不是双射函数，因为它不满足单射条件，例如当 $x=1$ 和 $x=-1$ 时，y 都等于 2。
- $y=2x+3$ 是双射函数。单射方面，若 $x_1\neq x_2$，则 $2x_1+3\neq 2x_2+3$；满射方面，对于任意实数 y，可以通过 $x=\dfrac{y-3}{2}$ 找到对应的 x 使得 $y=2x+3$，定义域和值域都是实数集 R。

计算反常积分$\int_{-\infty}^{0}\frac{e^{\frac{1}{x}}}{x^2}dx$。

1. **换元**：
   - 令$t=\frac{1}{x}$，则$dt=-\frac{1}{x^2}dx$，当$x=-\infty$时，$t=0$；当$x=0$时，$t=-\infty$。
   - 原积分$\int_{-\infty}^{0}\frac{e^{\frac{1}{x}}}{x^2}dx$可化为$\int_{0}^{-\infty}-e^{t}dt$。
2. **计算积分**：
   - 根据积分的基本公式$\int e^{t}dt=e^{t}+C$，对于$\int_{0}^{-\infty}-e^{t}dt$，先计算不定积分$-\int e^{t}dt=-e^{t}+C$。
   - 再计算定积分：
     - 由牛顿 - 莱布尼茨公式，$\int_{0}^{-\infty}-e^{t}dt=-e^{t}\Big|_{0}^{-\infty}$。
     - 计算$-\left(e^{-\infty}-e^{0}\right)$，因为$e^{-\infty}=\lim\limits_{t\to-\infty}e^{t}=0$，$e^{0}=1$，所以$-\left(e^{-\infty}-e^{0}\right)=-(0-1)=1$。

所以，反常积分$\int_{-\infty}^{0}\frac{e^{\frac{1}{x}}}{x^2}dx=1$。（原答案在换元后积分上下限处理有误，导致结果错误）

计算极限$\lim\limits_{x\to0}\dfrac{1-e^x}{x}$

解答

1. **方法一：等价无穷小替换**

   当$x\to0$时，$e^x-1\sim x$，那么$1-e^x=-(e^x-1)\sim -x$。

   所以$\lim\limits_{x\to0}\dfrac{1-e^x}{x}=\lim\limits_{x\to0}\dfrac{-x}{x}=-1$。

2. **方法二：洛 - 必达法则**

   对于$\lim\limits_{x\to0}\dfrac{1-e^x}{x}$，它是“$\dfrac{0}{0}$”型的极限（当$x=0$时，分子$1-e^0=1-1=0$，分母$x=0$）。

   根据洛 - 必达法则，若$\lim\limits_{x\to a}\dfrac{f(x)}{g(x)}$是“$\dfrac{0}{0}$”型或“$\dfrac{\infty}{\infty}$”型，那么$\lim\limits_{x\to a}\dfrac{f(x)}{g(x)}=\lim\limits_{x\to a}\dfrac{f'(x)}{g'(x)}$。

   对分子$1-e^x$求导，根据求导公式$(e^x)'=e^x$，$(C)'=0$（C为常数），可得$(1-e^x)'=-e^x$；对分母x求导，$(x)'=1$。

   所以$\lim\limits_{x\to0}\dfrac{1-e^x}{x}=\lim\limits_{x\to0}\dfrac{-e^x}{1}=\dfrac{-e^0}{1}=-1$。

综上，极限$\lim\limits_{x\to0}\dfrac{1-e^x}{x}$的结果为$-1$。

已知$\sin\theta=\frac{3}{5}$，且θ是第一象限角（补充条件使结果唯一），求$\tan\theta$的值。

因为$\sin\theta=\frac{3}{5}$，且$\sin^2\theta+\cos^2\theta=1$，所以

$\cos\theta=\pm\sqrt{1-\sin^2\theta}=\pm\sqrt{1-(\frac{3}{5})^2}=\pm\sqrt{1-\frac{9}{25}}=\pm\sqrt{\frac{16}{25}}=\pm\frac{4}{5}$。

又因为θ是第一象限角，在第一象限中余弦值为正，所以$\cos\theta=\frac{4}{5}$。

根据正切函数的定义$\tan\theta=\frac{\sin\theta}{\cos\theta}$，可得$\tan\theta=\frac{\frac{3}{5}}{\frac{4}{5}}=\frac{3}{4}$。

已知 T 是从 $\mathbb{R}^2$ 到 $\mathbb{R}^2$ 的线性变换，且 $T(1,2)=(1,3)$，$T(0,1)=(1,4)$，求 $T(3,-1)$。

1. **将向量$(3,-1)$用已知向量$(1,2)$和$(0,1)$线性表示**

   设 $(3,-1)=a(1,2)+b(0,1)$，根据向量相等的定义，可得方程组$\begin{cases}a\times1+b\times0=3\\a\times2+b\times1=-1\end{cases}$。

   由第一个方程可得 $a=3$，将 $a=3$ 代入第二个方程 $2a+b=-1$ 中，即 $2\times3+b=-1$，解得 $b=-1-6=-7$。

   所以 $(3,-1)=3(1,2)-7(0,1)$。

2. **利用线性变换的性质计算$T(3,-1)$**

因为 T 是线性变换，根据线性变换的性质：对于任意向量 $\vec{u},\vec{v}\in\mathbb{R}^2$ 和实数 k_1,k_2，有 $T(k_1\vec{u}+k_2\vec{v})=k_1T(\vec{u})+k_2T(\vec{v})$。

已知 $T(1,2)=(1,3)$，$T(0,1)=(1,4)$，那么 $T(3,-1)=T(3(1,2)-7(0,1))$。

由线性变换性质可得：

$T(3(1,2)-7(0,1))=3T(1,2)-7T(0,1)$。

计算 $3T(1,2)-7T(0,1)$：

$3T(1,2)=3(1,3)=(3\times1,3\times3)=(3,9)$；

$7T(0,1)=7(1,4)=(7\times1,7\times4)=(7,28)$。

则 $3T(1,2)-7T(0,1)=(3-7,9-28)=(-4,-19)$。

所以 $T(3,-1)=(-4,-19)$。

已知函数$f(x)=\begin{cases}3x-2, & x\leq 3\\ mx+1, & x>3\end{cases}$在$x=3$处连续，求$m$的值。

函数$f(x)$在$x=a$处连续的充要条件是$\lim\limits_{x\to a^-}f(x)=\lim\limits_{x\to a^+}f(x)=f(a)$，这里$a=3$。

1. **计算$\lim\limits_{x\to 3^-}f(x)$（即$x$从左侧趋近于3时$f(x)$的极限）**：

   当$x\to 3^-$时，$f(x)=3x-2$，将$x=3$代入$3x-2$可得：$\lim\limits_{x\to 3^-}f(x)=3\times 3-2=9-2=7$。

2. **计算$\lim\limits_{x\to 3^+}f(x)$（即$x$从右侧趋近于3时$f(x)$的极限）**：

   当$x\to 3^+$时，$f(x)=mx+1$，则$\lim\limits_{x\to 3^+}f(x)=3m+1$。

3. **计算$f(3)$**：

   当$x=3$时，代入$f(x)=3x-2$（因为$x=3$满足$x\leq 3$这一段的表达式），$f(3)=3\times 3-2=7$。

因为函数$f(x)$在$x=3$处连续，所以$\lim\limits_{x\to 3^-}f(x)=\lim\limits_{x\to 3^+}f(x)$，即$3m+1=7$。

移项可得$3m=7-1=6$，解得$m=2$。

综上，m的值为2。

已知函数$f(x)=|x^2-3x+2|+|x-1|$，分别写出$f(x)$在$x\leq 1$和$[1,2]$上的表达式。

1. **分析绝对值内式子的正负性**

   对于$y=x^2-3x+2$，令$y=0$，即$x^2-3x+2=0$，因式分解得$(x-1)(x-2)=0$，解得$x=1$或$x=2$。

   二次函数$y=x^2-3x+2$的二次项系数大于0，图象开口向上，所以当$x\leq 1$或$x\geq 2$时，$x^2-3x+2\geq 0$；当$1<x<2$时，$x^2-3x+2<0$。

   对于$y=x-1$，当$x\geq 1$时，$x-1\geq 0$；当$x<1$时，$x-1<0$。

2. **当$x\leq 1$时求$f(x)$的表达式**

   此时$x^2-3x+2\geq 0$，$x-1\leq 0$，则

   $f(x)=|x^2-3x+2|+|x-1|=(x^2-3x+2)-(x-1)$。

   对$(x^2-3x+2)-(x-1)$进行化简：

   $$\begin{aligned}&x^2-3x+2-x+1\\=&x^2-4x+3\end{aligned}$$

3. **当$x \in [1,2]$时求$f(x)$的表达式**

   此时$x^2-3x+2 \leq 0$，$x-1 \geq 0$，则

   $f(x)=|x^2-3x+2|+|x-1|=-(x^2-3x+2)+(x-1)$。

   对$-(x^2-3x+2)+(x-1)$进行化简：

$$
\begin{aligned}
&-x^2+3x-2+x-1 \\
=&-x^2+4x-3
\end{aligned}
$$

综上，$f(x)$在$x \leq 1$上的表达式为x^2-4x+3；在$[1,2]$上的表达式为$-x^2+4x-3$。

概率论与数理统计

某医院平均每小时出生的婴儿数服从参数为 $\lambda = 3$ 的泊松分布。求在某一小时内，该医院出生婴儿数不超过 2 个的概率。

设 X 表示该医院每小时出生的婴儿数，已知 $X \sim P(3)$，泊松分布的概率质量函数为

$P(X = k) = \dfrac{e^{-\lambda}\lambda^k}{k!}$，$k = 0, 1, 2, \cdots$，其中 λ 为参数，$e \approx 2.71828$。

要求 $P(X \leq 2)$，即 $P(X \leq 2) = P(X = 0) + P(X = 1) + P(X = 2)$。

- 当 $k = 0$ 时：

  $P(X = 0) = \dfrac{e^{-3} \times 3^0}{0!} = e^{-3} \approx 0.0498$。

- 当 $k = 1$ 时：

  $P(X = 1) = \dfrac{e^{-3} \times 3^1}{1!} = 3e^{-3} \approx 3 \times 0.0498 = 0.1494$。

- 当 $k = 2$ 时：

  $P(X = 2) = \dfrac{e^{-3} \times 3^2}{2!} = \dfrac{9e^{-3}}{2} \approx \dfrac{9 \times 0.0498}{2} = 0.2241$。

  所以 $P(X \leq 2) = 0.0498 + 0.1494 + 0.2241 = 0.4233$。

  即该医院在某一小时内出生婴儿数不超过 2 个的概率为 0.4233。

已知某班学生的数学考试成绩 X 服从正态分布 $N(75, 100)$（其中均值 $\mu = 75$，方差 $\sigma^2 = 100$，则标准差 $\sigma = 10$）。求该班学生数学成绩在 65 到 85 之间的概率。（参考数据：$\Phi(1) = 0.8413$，Φ 为标准正态分布的分布函数）

本题可先将给定的正态分布转化为标准正态分布，再利用标准正态分布的性质和参考数据求解。

步骤一：明确已知条件

已知成绩X服从正态分布$N(75, 100)$，即均值$\mu = 75$，标准差$\sigma = \sqrt{100} = 10$。

要求$P(65 < X < 85)$。

步骤二：进行标准化变换

设$Z = \dfrac{X - \mu}{\sigma}$，$Z$服从标准正态分布$N(0, 1)$。

当$X = 65$时，$Z_1 = \dfrac{65 - 75}{10} = \dfrac{-10}{10} = -1$；

当$X = 85$时，$Z_2 = \dfrac{85 - 75}{10} = \dfrac{10}{10} = 1$。

则$P(65 < X < 85) = P(-1 < \dfrac{X - 75}{10} < 1) = P(-1 < Z < 1)$。

步骤三：利用标准正态分布的性质计算概率

根据标准正态分布的性质$\Phi(-z) = 1 - \Phi(z)$以及$P(-1 < Z < 1) = \Phi(1) - \Phi(-1)$。

因为$\Phi(-1) = 1 - \Phi(1)$，已知$\Phi(1) = 0.8413$，所以$\Phi(-1) = 1 - 0.8413 = 0.1587$。

则$P(-1 < Z < 1) = \Phi(1) - \Phi(-1) = 0.8413 - 0.1587 = 0.6826$。

综上，该班学生数学成绩在65到85之间的概率为0.6826。

两个人随机坐在一个有 12 个位置的圆桌旁，求两个人相邻的概率是多少？

1. **计算基本事件总数**：
   第一个人在 12 个位置中任意选一个坐下，有 12 种坐法；当第一个人坐下后，第二个人在剩下的 11 个位置中选一个坐下，有 11 种坐法。
   根据分步乘法计数原理，两人随机坐的总情况数为$n = 12\times11$种。但因为是圆桌排列，对于每一种坐法，整体旋转后是同一种相对位置关系，所以实际上不同的坐法有$A_{12}^{2}\div12 = 11$种（这一步也可以这样理解：固定第一个人位置，只考虑第二个人的选择，第二个人有 11 种选择）。
2. **计算两人相邻的情况数**：
   当第一个人坐下后，与第一个人相邻的位置有 2 个，所以第二个人坐在与第一个人相邻位置的情况有$m = 2$种。
3. **计算概率**：
   根据古典概型概率公式$P(A)=\frac{m}{n}$，这里$n = 11$（不同的相对坐法总数），$m = 2$（两人相邻的情况数），所以两人相邻的概率$P=\frac{2}{11}$。

已知随机变量 X 的概率密度函数为 $f(x)=\begin{cases}k, & 1\leq x\leq 3\\ 0, & \text{otherwise}\end{cases}$，求以下内容：

1. 常数 k 的值；
2. 分布函数 $F(x)$；
3. 期望 $E(X)$；
4. $P(X\geq 1.5)$；
5. 方差 $Var(X)$。

1. **求常数 k 的值**

   根据概率密度函数的性质 $\int_{-\infty}^{\infty} f(x)dx=1$，因为 $f(x)$ 仅在 $1\leq x\leq 3$ 时不为 0，所以有：

$$\begin{aligned}\int_1^3 k dx &= 1\\ k\int_1^3 dx &= 1\\ k(x)\Big|_1^3 &= 1\\ k(3-1) &= 1\\ 2k &= 1\\ k &= \frac{1}{2}\end{aligned}$$

2. **求分布函数 $F(x)$**

- 当 $x \leq 1$ 时，$F(x) = \int_{-\infty}^{x} f(t)dt = \int_{-\infty}^{x} 0dt = 0$；
- 当 $1 < x \leq 3$ 时，$F(x) = \int_{-\infty}^{1} f(t)dt + \int_{1}^{x} f(t)dt = 0 + \int_{1}^{x} \frac{1}{2}dt = \frac{1}{2}(x-1) = \frac{1}{2}x - \frac{1}{2}$；
- 当 $x > 3$ 时，

  $F(x) = \int_{-\infty}^{1} f(t)dt + \int_{1}^{3} f(t)dt + \int_{3}^{x} f(t)dt = 0 + \int_{1}^{3} \frac{1}{2}dt + 0 = \frac{1}{2}(3-1) = 1$。

  所以 $F(x) = \begin{cases} 0, & x \leq 1 \\ \frac{1}{2}x - \frac{1}{2}, & 1 < x \leq 3 \\ 1, & x > 3 \end{cases}$。

3. **求期望 $E(X)$**

   根据期望的计算公式 $E(X)=\int_{-\infty}^{\infty}xf(x)dx=\int_{1}^{3}x\cdot\frac{1}{2}dx$ 。

$$\begin{aligned}\int_{1}^{3}\frac{1}{2}xdx&=\frac{1}{2}\cdot\frac{1}{2}x^{2}\Big|_{1}^{3}\\&=\frac{1}{4}(3^{2}-1^{2})\\&=\frac{1}{4}(9-1)\\&=2\end{aligned}$$

4. **求 $P(X\geq 1.5)$**

$$\begin{aligned}P(X\geq 1.5)&=1-P(X<1.5)\\&=1-F(1.5)\\&=1-(\frac{1}{2}\times 1.5-\frac{1}{2})\\&=1-(\frac{3}{4}-\frac{2}{4})\\&=0.75\end{aligned}$$

5. **求方差 $Var(X)$**

先求 $E(X^2)$，$E(X^2)=\int_{-\infty}^{\infty}x^2f(x)dx=\int_1^3 x^2\cdot\frac{1}{2}dx$。

$$\begin{aligned}\int_1^3\frac{1}{2}x^2dx&=\frac{1}{2}\cdot\frac{1}{3}x^3\Big|_1^3\\&=\frac{1}{6}(3^3-1^3)\\&=\frac{1}{6}(27-1)\\&=\frac{13}{3}\end{aligned}$$

根据方差公式 $Var(X)=E(X^2)-[E(X)]^2$，

$Var(X)=\frac{13}{3}-2^2=\frac{13}{3}-4=\frac{13-12}{3}=\frac{1}{3}\neq 0$。

将 10 拆成不同正整数相加，有几种方法?

解答

我们可以通过列举的方法来找出所有的组合：

1. 从最小的正整数 1 开始考虑：
   - $1+9=10$;
   - $1+2+7=10$;
   - $1+2+3+4=10$;
   - $1+3+6=10$;
   - $2+8=10$;
   - $2+3+5=10$;
   - $4+6=10$ 。

从 1 - 100 中选一个数，求这个数为质数的概率。

解答

1. **确定 1 - 100 中的质数个数**
   质数是指在大于 1 的自然数中，除了 1 和它自身外，不能被其他自然数整除的数。
   1 - 100 中的质数有：2、3、5、7、11、13、17、19、23、29、31、37、41、43、47、53、59、61、67、71、73、79、83、89、97，共 25 个。
2. **计算概率**
   从 1 - 100 中选一个数，总共有 100 种选法，而选到质数的情况有 25 种。
   根据古典概型概率公式 $P(A)=\frac{m}{n}$（其中 m 是事件 A 包含的基本事件个数，n 是基本事件总数），可得选到质数的概率 $P=\frac{25}{100}=\frac{1}{4}$。

已知 $y = Ce^{4x} + De^{5x}$，下列哪个方程成立？

① $\frac{d^2y}{dx^2} + 9\frac{dy}{dx} + 20y = 0$

② $\frac{d^2y}{dx^2} - 9\frac{dy}{dx} + 20y = 0$

③ $\frac{d^2y}{dx^2} + 5\frac{dy}{dx} + 4y = 0$

④ $\frac{d^2y}{dx^2} - 5\frac{dy}{dx} + 4y = 0$

1. **求一阶导数 y'**：

   根据求导公式 $(e^{ax})' = ae^{ax}$ 以及加法求导法则 $(u+v)' = u' + v'$，对 $y = Ce^{4x} + De^{5x}$ 求导，可得 $y' = 4Ce^{4x} + 5De^{5x}$。

2. **求二阶导数 y''**：

   再次对 y' 求导，$y'' = (4Ce^{4x} + 5De^{5x})' = 16Ce^{4x} + 25De^{5x}$。

3. **将 y、y'、y'' 代入各个方程进行验证**：

- 对于方程①：把 y、y'、y'' 代入 $\frac{d^2y}{dx^2}+9\frac{dy}{dx}+20y$，计算

  $(16Ce^{4x}+25De^{5x})+9(4Ce^{4x}+5De^{5x})+20(Ce^{4x}+De^{5x})$，看是否等于 0。

- 对于方程②：把 y、y'、y'' 代入 $\frac{d^2y}{dx^2}-9\frac{dy}{dx}+20y$，计算

  $(16Ce^{4x}+25De^{5x})-9(4Ce^{4x}+5De^{5x})+20(Ce^{4x}+De^{5x})$，看是否等于 0。

- 对于方程③：把 y、y'、y'' 代入 $\frac{d^2y}{dx^2}+5\frac{dy}{dx}+4y$，计算

  $(16Ce^{4x}+25De^{5x})+5(4Ce^{4x}+5De^{5x})+4(Ce^{4x}+De^{5x})$，看是否等于 0。

- 对于方程④：把 y、y'、y'' 代入 $\frac{d^2y}{dx^2}-5\frac{dy}{dx}+4y$，计算

  $(16Ce^{4x}+25De^{5x})-5(4Ce^{4x}+5De^{5x})+4(Ce^{4x}+De^{5x})$，看是否等于 0。通过计算可知，代入方程②时等式成立。

已知随机变量 X 的概率密度函数为 $f(x)=\begin{cases}4x^n, & 0\le x\le 1\\ 0, & \text{otherwise}\end{cases}$，先确定常数 n 的值（使得 $f(x)$ 满足概率密度函数的性质），再求 $F(\frac{1}{3})$，其中 $F(x)$ 是 X 的分布函数。

1. **确定常数 n 的值**

   根据概率密度函数的性质 $\int_{-\infty}^{\infty} f(x)dx=1$，因为 $f(x)$ 仅在 $0\le x\le 1$ 时不为 0，所以有：

$$\begin{aligned}\int_0^1 4x^n dx&=1\\ 4\times\frac{x^{n+1}}{n+1}\Big|_0^1&=1\\ \frac{4}{n+1}(1^{n+1}-0^{n+1})&=1\\ \frac{4}{n+1}&=1\\ n+1&=4\\ n&=3\end{aligned}$$

   此时概率密度函数为 $f(x)=\begin{cases}4x^3, & 0\le x\le 1\\ 0, & \text{otherwise}\end{cases}$。

2. **求分布函数 $F(x)$**

   分布函数 $F(x)=\int_{-\infty}^{x}f(t)dt$。

- 当 $x<0$ 时，$F(x)=\int_{-\infty}^{x}0dt=0$。
- 当 $0\le x\le 1$ 时，$F(x)=\int_{-\infty}^{0}0dt+\int_{0}^{x}4t^3dt=0+t^4\Big|_0^x=x^4$。
- 当 $x>1$ 时，$F(x)=\int_{-\infty}^{0}0dt+\int_{0}^{1}4t^3dt+\int_{1}^{x}0dt=0+t^4\Big|_0^1+0=1$。

   所以 $F(x)=\begin{cases}0, & x<0\\ x^4, & 0\le x\le 1\\ 1, & x>1\end{cases}$。

3. **求 $F(\frac{1}{3})$**

   因为 $0\le\frac{1}{3}\le 1$，将 $x=\frac{1}{3}$ 代入 $F(x)=x^4$，可得 $F(\frac{1}{3})=(\frac{1}{3})^4=\frac{1}{81}$。

有 30 个橙子，其中 6 个是坏的。随机从中选取 6 个橙子，求选取的 6 个橙子中没有坏橙子的概率。

There are 30 oranges, 6 of which are bad. Randomly select 6 oranges from them. Find the probability that there are no bad oranges among the 6 selected oranges.

本题可根据古典概型的概率公式，结合组合数的计算方法来求解。

1. **计算从30个橙子中选取6个橙子的总组合数**

   从n个不同元素中取出m个元素的组合数记为C_n^m，其计算公式为$C_n^m = \frac{n!}{m!(n-m)!}$。

   从30个橙子中选取6个橙子的总组合数为$C_{30}^6 = \frac{30!}{6!(30-6)!} = \frac{30!}{6!\times 24!}$。

2. **计算从24个好橙子中选取6个橙子的组合数**

   好橙子的数量为$30-6=24$个，从24个好橙子中选取6个橙子的组合数为

   $C_{24}^6 = \frac{24!}{6!(24-6)!} = \frac{24!}{6!\times 18!}$。

3. **根据古典概型概率公式计算概率**

古典概型概率公式为$P(A) = \frac{m}{n}$，其中n是基本事件总数，m是事件A所包含的基本事件数。

设事件A为“选取的6个橙子中没有坏橙子”，则$P(A) = \frac{C_{24}^6}{C_{30}^6} = \frac{\frac{24!}{6! \times 18!}}{\frac{30!}{6! \times 24!}} = \frac{24! \times 24!}{18! \times 30!}$

$$
\begin{aligned}
&= \frac{24 \times 23 \times 22 \times 21 \times 20 \times 19}{30 \times 29 \times 28 \times 27 \times 26 \times 25} \\
&= \frac{1307504}{4275180} \\
&\approx 0.306
\end{aligned}
$$

综上，选取的6个橙子中没有坏橙子的概率约为0.306。

线性代数

已知矩阵$M_{2\times3}$，$N_{3\times5}$，判断以下说法是否正确：

1. $MN=NM$
2. $MN=(M^TN^T)^T$
3. $MN=0$
4. MN是2×5的矩阵

1. **对于$MN = NM$**：
   - 矩阵乘法要求前一个矩阵的列数等于后一个矩阵的行数。M是2×3矩阵，N是3×5矩阵，MN可以相乘，结果是2×5矩阵。
   - 而N是3×5矩阵，M是2×3矩阵，NM不满足矩阵乘法规则（N的列数5不等于M的行数2），不能相乘，所以$MN = NM$错误。
2. **对于$MN = (M^T N^T)^T$**：
   - 根据矩阵转置的性质$(AB)^T = B^T A^T$。对于MN，$(MN)^T = N^T M^T$，而$(M^T N^T)^T$与$(MN)^T$不相等（顺序错误），所以$MN = (M^T N^T)^T$错误。
3. **对于$MN = 0$**：
   - 一般情况下，仅知道矩阵M和N的维度，并不能得出它们相乘结果为零矩阵的结论，矩阵相乘结果取决于矩阵的元素值，所以$MN = 0$错误。
4. **对于MN是2×5的矩阵**：
   - 矩阵乘法中，若A是$m \times n$矩阵，B是$n \times p$矩阵，那么AB是$m \times p$矩阵。这里M是2×3矩阵，N是3×5矩阵，所以MN是2×5的矩阵，该说法正确。

判断线性方程组$\begin{pmatrix}1 & 2 & 3 & 4\\0 & 0 & 0 & 5\\0 & 0 & 0 & 0\end{pmatrix}$解的情况。

解答

对于增广矩阵$\begin{pmatrix}1 & 2 & 3 & 4\\0 & 0 & 0 & 5\\0 & 0 & 0 & 0\end{pmatrix}$，它对应的线性方程组为：

$$\begin{cases}x_1 + 2x_2 + 3x_3 = 4\\0x_1 + 0x_2 + 0x_3 = 5\\0x_1 + 0x_2 + 0x_3 = 0\end{cases}$$

在第二个方程$0x_1 + 0x_2 + 0x_3 = 5$中，左边恒为0，右边为5，此方程不成立。

根据线性方程组解的判定规则，当增广矩阵对应的线性方程组中出现矛盾方程（如$0 = a$，$a \neq 0$）时，方程组无解。

所以该线性方程组无解。

在什么情况下，$Var(X+Y)=Var(X)+Var(Y)$?

根据方差的性质：

$$\begin{aligned} Var(X+Y) &= E[(X+Y)-E(X+Y)]^2 \\ &= E[(X-E(X)+Y-E(Y))^2] \\ &= E[(X-E(X))^2+(Y-E(Y))^2+2(X-E(X))(Y-E(Y))] \\ &= Var(X)+Var(Y)+2Cov(X,Y) \end{aligned}$$

其中$Cov(X,Y)=E[(X-E(X))(Y-E(Y))]$为$X$与$Y$的协方差。

当X和Y相互独立时，$Cov(X,Y)=0$。

将$Cov(X,Y)=0$代入$Var(X+Y)=Var(X)+Var(Y)+2Cov(X,Y)$中，可得

$Var(X+Y)=Var(X)+Var(Y)$。

所以当随机变量X和Y相互独立时，$Var(X+Y)=Var(X)+Var(Y)$。

设向量空间V是由向量$\alpha_1 = (1,1,0)^T$，$\alpha_2 = (1,0,1)^T$，$\alpha_3 = (0,1,1)^T$生成的三维向量空间$\mathbb{R}^3$的子空间。判断向量$\beta = (2,1,1)^T$是否属于V，若属于，求出β在基$\{\alpha_1, \alpha_2, \alpha_3\}$下的坐标。

1. **判断β是否属于V**

   设存在实数x, y, z，使得$\beta = x\alpha_1 + y\alpha_2 + z\alpha_3$，即

   $$\begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix} = x \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} + y \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + z \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} x+y \\ x+z \\ y+z \end{pmatrix}$$

   由此得到方程组$\begin{cases} x + y = 2 \\ x + z = 1 \\ y + z = 1 \end{cases}$

   将第一个方程$x + y = 2$变形为$x = 2 - y$，代入第二个方程$x + z = 1$中，得到$2 - y + z = 1$，即$z = y - 1$。

   把$z = y - 1$代入第三个方程$y + z = 1$，有$y + (y - 1) = 1$，$2y = 2$，解得$y = 1$。

   因为$y = 1$，所以$x = 2 - y = 1$，$z = y - 1 = 0$。

   由于存在实数$x = 1, y = 1, z = 0$使得$\beta = x\alpha_1 + y\alpha_2 + z\alpha_3$成立，所以$\beta$属于$V$。

2. **求β在基$\{\alpha_1, \alpha_2, \alpha_3\}$下的坐标**

   由上述计算可知，β在基$\{\alpha_1, \alpha_2, \alpha_3\}$下的坐标为$(1,1,0)$。

Question

Given the linear - equation system $\begin{cases} ax+by=f \\ cx+dy=g \end{cases}$, and its augmented matrix is $\begin{pmatrix} a & b & f \\ c & d & g \end{pmatrix}$. After elementary row operations, it is transformed into $\begin{pmatrix} a & b & f \\ 0 & \frac{ad-bc}{a} & \frac{ag-cf}{a} \end{pmatrix}$ $(a \neq 0)$. Determine the truth or falsehood of the following four propositions:

① When $f=g=0$, $x=y=0$ is a solution of the equation system.

② When $ad-bc=0$, the equation system has no solution.

③ When $ag-cf=0$, the equation system has no solution.

④ When $c=d=g=0$, the equation system has no solution.

已知线性方程组$\begin{cases} ax+by=f \\ cx+dy=g \end{cases}$，其增广矩阵为$\begin{pmatrix} a & b & f \\ c & d & g \end{pmatrix}$，经过初等行变换得到$\begin{pmatrix} a & b & f \\ 0 & \frac{ad-bc}{a} & \frac{ag-cf}{a} \end{pmatrix}$（$a \neq 0$），判断以下四个命题的真假：

①当$f=g=0$时，$x=y=0$为方程组的解。

②当$ad-bc=0$时，方程组无解。

③当$ag-cf=0$时，方程组无解。

④当$c=d=g=0$时，方程组无解。

解答

1. **判断命题①**

   当$f=g=0$时，原方程组变为$\begin{cases}ax+by=0\\cx+dy=0\end{cases}$。

   若$ad-bc\neq 0$，方程组有唯一解$x=y=0$；若$ad-bc=0$，方程组可能有无穷多解，不一定只有$x=y=0$这一组解，所以命题①错误。

2. **判断命题②**

   当$ad-bc=0$时：

   若$\frac{ag-cf}{a}=0$（即$ag-cf=0$），此时方程组有无穷多解；若$\frac{ag-cf}{a}\neq 0$，方程组无解。所以仅$ad-bc=0$不能得出方程组无解，命题②错误。

3. **判断命题③**

   当$ag-cf=0$时，若$ad-bc\neq 0$，方程组有唯一解；若$ad-bc=0$，方程组有无穷多解。所以仅$ag-cf=0$不能得出方程组无解，命题③错误。

4. **判断命题④**

   当$c=d=g=0$时，原方程组变为$\begin{cases}ax+by=f\\0x+0y=0\end{cases}$。

   若$a\neq 0$，$x=\frac{f-by}{a}$，y可以取任意实数，方程组有无穷多解；若$a=0$且$b\neq 0$，$y=\frac{f}{b}$，x可以取任意实数，方程组也有无穷多解。所以命题④错误。

已知向量$\vec{a}=(1,2,3)$，$\vec{b}=(4,5,6)$，求$\vec{a}\cdot\vec{b}$（向量$\vec{a}$与向量$\vec{b}$的数量积）。

根据向量数量积的坐标运算公式，若$\vec{m}=(x_1,y_1,z_1)$，$\vec{n}=(x_2,y_2,z_2)$，则

$\vec{m}\cdot\vec{n}=x_1x_2+y_1y_2+z_1z_2$。

对于向量$\vec{a}=(1,2,3)$，$\vec{b}=(4,5,6)$，可得：

$\vec{a}\cdot\vec{b}=1\times4+2\times5+3\times6=4+10+18=32$。

已知矩阵A是一个$n \times n$的矩阵，λ为非零实数。判断下列说法正确的是（ ）

A. 若A是对称矩阵且$\lambda > 0$，则λA是非对称矩阵。

B. 若A是反对称矩阵且$\lambda > 0$，则λA是反对称矩阵。

C. 若A是对称矩阵且$\lambda < 0$，则λA是反对称矩阵。

D. 若A是反对称矩阵且$\lambda < 0$，则λA是对称矩阵。

1. **对称矩阵和反对称矩阵的定义**
   - 对于$n \times n$矩阵$A = (a_{ij})$，若$A^T = A$，即$a_{ij} = a_{ji}$（$i, j = 1, 2, \cdots, n$），则称A为对称矩阵。
   - 若$A^T = -A$，即$a_{ij} = -a_{ji}$（$i, j = 1, 2, \cdots, n$），则称A为反对称矩阵。
2. **对选项 A 的分析**
   已知A是对称矩阵，即$A^T = A$。对于λA，$(\lambda A)^T = \lambda A^T$（根据矩阵数乘的转置性质：$(kA)^T = kA^T$，$k$为实数），因为$A^T = A$，所以$(\lambda A)^T = \lambda A$，这表明当$\lambda > 0$时，$\lambda A$也是对称矩阵，A 选项**错误**。

3. **对选项 B 的分析**
   已知A是反对称矩阵，即$A^T = -A$。对于λA，$(\lambda A)^T = \lambda A^T$，又因为$A^T = -A$，所以$(\lambda A)^T = \lambda(-A) = -(\lambda A)$，这说明当$\lambda > 0$时，$\lambda A$是反对称矩阵，B 选项**正确**。

4. **对选项 C 的分析**
   已知A是对称矩阵，即$A^T = A$。对于λA，$(\lambda A)^T = \lambda A^T = \lambda A$，因为$\lambda < 0$，但$(\lambda A)^T = \lambda A$，不满足反对称矩阵$(\lambda A)^T = -(\lambda A)$的条件，所以$\lambda A$不是反对称矩阵，C 选项**错误**。

5. **对选项 D 的分析**
   已知A是反对称矩阵，即$A^T = -A$。对于λA，$(\lambda A)^T = \lambda A^T$，因为$A^T = -A$，所以$(\lambda A)^T = \lambda(-A) = -(\lambda A)$，这表明$\lambda A$是反对称矩阵，而不是对称矩阵，D 选项**错误**。

综上，答案是 B。

已知A是$n \times n$矩阵，λ是A的一个特征值，求$A^2 + I$（I为n阶单位矩阵）的特征值是多少?

Given that A is an $n \times n$ matrix and λ is an eigenvalue of A, what is the eigenvalue of $A^2 + I$ (where I is the $n-$order identity matrix)?

步骤一：明确特征值和特征向量的定义

设A是n阶矩阵，若存在数λ和非零n维列向量ξ，使得$A\xi = \lambda\xi$，则称λ是矩阵A的特征值，ξ是矩阵A属于特征值λ的特征向量。

步骤二：推导$A^2\xi$与ξ的关系

已知$A\xi = \lambda\xi$，等式两边同时左乘矩阵A，可得：

$$A(A\xi) = A(\lambda\xi)$$

根据矩阵乘法的结合律$A(A\xi) = A^2\xi$，以及数乘矩阵的性质$A(\lambda\xi) = \lambda(A\xi)$，又因为$A\xi = \lambda\xi$，所以$A(\lambda\xi) = \lambda(\lambda\xi) = \lambda^2\xi$。

因此，$A^2\xi = \lambda^2\xi$。

步骤三：计算$(A^2+I)\xi$

根据矩阵运算的分配律可得：

$$(A^2+I)\xi = A^2\xi + I\xi$$

因为I为n阶单位矩阵，对于任意n维向量ξ，都有$I\xi = \xi$，且由步骤二可知$A^2\xi = \lambda^2\xi$，所以：

$$(A^2+I)\xi = \lambda^2\xi + \xi = (\lambda^2+1)\xi$$

步骤四：确定A^2+I的特征值

由特征值和特征向量的定义可知，对于矩阵$B = A^2+I$，存在非零向量ξ，使得$B\xi = (\lambda^2+1)\xi$，所以A^2+I的一个特征值是λ^2+1。

综上，若λ是矩阵A的一个特征值，那么A^2+I的特征值为λ^2+1。

程序部分

编写一个 Python 程序，输入一个月份，判断这个月有多少天。其中 2 月较为特殊，需要根据是否是闰年判断天数（闰年 2 月 29 天，平年 2 月 28 天，闰年判断规则同上述题目：世纪年能被 400 整除，非世纪年能被 4 整除的为闰年），1、3、5、7、8、10、12 月有 31 天，4、6、9、11 月有 30 天。

```
month = int(input("请输入一个月份(1 - 12): "))
year = int(input("请输入年份: "))
if month in [1, 3, 5, 7, 8, 10, 12]:
    days = 31
elif month in [4, 6, 9, 11]:
    days = 30
elif month == 2:
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        days = 29
    else:
        days = 28
print(f"{year}年{month}月有{days}天")
```

```c
#include <stdio.h>

int main() {
    int month, year, days;
    printf("请输入一个月份(1 - 12): ");
    scanf("%d", &month);
    printf("请输入年份: ");
    scanf("%d", &year);

    if (month == 1 || month == 3 || month == 5 || month == 7 || month == 8 || month == 10 || month == 12) {
        days = 31;
    } else if (month == 4 || month == 6 || month == 9 || month == 11) {
        days = 30;
    } else if (month == 2) {
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
            days = 29;
        } else {
            days = 28;
        }
    }

    printf("%d年%d月有%d天\n", year, month, days);

    return 0;
}
```

编写一个程序，输入一个正整数，将其各位数字的平方相加，若结果等于 1 则停止运算；若结果不等于 1 ，则将该结果作为新的数，继续重复上述各位数字平方相加的操作，直到结果等于 1 为止。如果永远无法得到 1 ，程序应持续循环（在实际编程中，可设定一个较大的循环次数上限来模拟 “永远无法得到 1” 的情况）。

```
num = int(input("请输入一个正整数: "))
while True:
    sum_square = 0
    for digit in str(num):
        sum_square += int(digit) ** 2
    if sum_square == 1:
        print("经过运算得到了1")
        break
    else:
        num = sum_square
```

Problem 1: Find elements larger than their four surrounding elements in a 2D array

Question: Given a 2D array, write a program to find all the elements that are larger than their four surrounding elements (up, down, left, and right, assuming valid indices within the array boundaries). For example, if we have a 2D array `arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`, we need to check each element in the array to see if it meets the criteria.

```python
def find_larger_than_surrounding(arr):
    rows = len(arr)
    cols = len(arr[0])
    result = []
    for i in range(rows):
        for j in range(cols):
            is_larger = True
            if i > 0 and arr[i][j] <= arr[i - 1][j]:
                is_larger = False
            if i < rows - 1 and arr[i][j] <= arr[i + 1][j]:
                is_larger = False
            if j > 0 and arr[i][j] <= arr[i][j - 1]:
                is_larger = False
            if j < cols - 1 and arr[i][j] <= arr[i][j + 1]:
                is_larger = False
            if is_larger:
                result.append(arr[i][j])
    return result


arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(find_larger_than_surrounding(arr))
```

Question 1: Find the second smallest number without using sorting.

Answer: One way to do this is to first find the minimum number in the array by iterating through it once and keeping track of the minimum value. Then, initialize a variable to store the second smallest number with a very large value (say, `Integer.MAX_VALUE` in Java). Iterate through the array again, and if the current number is not equal to the minimum number we found earlier and is smaller than the current second smallest number, update the second smallest number. Here is the Java code example:

```
def find_second_smallest(arr):
    # 找到最小的数
    min_num = float('inf')
    for num in arr:
        if num < min_num:
            min_num = num

    # 找到第二小的数
    second_smallest = float('inf')
    for num in arr:
        if num!= min_num and num < second_smallest:
            second_smallest = num
    return second_smallest
```

编写一个用 C、C++、Java 或 Python 语言实现的程序，该程序输入一个包含 100 个单词的数组，并按照字母顺序输出排序后的单词。评判将依据程序的正确性、效率和内存使用情况。

```
words = []
print("请输入100个单词：")
for _ in range(100):
    word = input()
    words.append(word)
sorted_words = sorted(words)
print("排序后的单词为：")
for word in sorted_words:
    print(word)
```

```
#include <stdio.h>
#include <string.h>

#define MAX_WORDS 100
#define MAX_LENGTH 100

void sortWords(char words[MAX_WORDS][MAX_LENGTH]) {
    int i, j;
    char temp[MAX_LENGTH];
    for (i = 0; i < MAX_WORDS - 1; i++) {
        for (j = i + 1; j < MAX_WORDS; j++) {
            if (strcmp(words[i], words[j]) > 0) {
                strcpy(temp, words[i]);
                strcpy(words[i], words[j]);
                strcpy(words[j], temp);
            }
        }
    }
}

int main() {
    char words[MAX_WORDS][MAX_LENGTH];
    int i;
```

编写一个 C++ 程序，该程序接收用户输入的一串字符串，程序需要遍历该字符串，识别其中的数字字符，并将这些数字字符转换为对应的数值后累加起来，最后输出这些数字的总和。注意，输入字符串中的数字字符是指 '0' - '9' ，且单个数字字符 '0' 不计入总和（如 "0123" 中，只计算 1、2、3 的和）。

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[1000];
    int sum = 0;
    printf("请输入一串字符串: ");
    scanf("%s", str);

    int len = strlen(str);
    for (int i = 0; i < len; i++) {
        if (str[i] >= '1' && str[i] <= '9') {
            sum += str[i] - '0';
        }
    }

    printf("数字字符的总和是: %d\n", sum);

    return 0;
}
```

编写一个 C++ 程序，该程序接收用户输入的一串字符串，程序需要遍历该字符串，识别其中的数字字符，并将这些数字字符转换为对应的数值后累加起来，最后输出这些数字的总和。注意，输入字符串中的数字字符是指 '0' - '9'，且单个数字字符 '0' 不计入总和（如 "0123" 中，只计算 1、2、3 的和）。

```
input_str = input("请输入一个字符串: ")
digit_sum = 0

for char in input_str:
    if char.isdigit() and char != '0':
        digit_sum = digit_sum + int(char)

print("字符串中数字的和为:", digit_sum)
```

给定一个正整数 `m` 代表金条的数量，每次可以选择拿走 1 个或者 2 个金条。要求编写一个程序，找出所有不同的拿完这些金条的方案，并将这些方案以列表的形式输出，每个方案也是一个列表，列表中的元素为每次拿走金条的数量。

示例

- 输入：`3`
  输出：`[[1, 1, 1], [1, 2], [2, 1]]`
- 输入：`2`
  输出：`[[1, 1], [2]]`

```python
def find_all_schemes(m):
    # 初始化一个列表，用于存储不同金条数量下的所有方案
    all_schemes = [[] for _ in range(m + 1)]
    # 当金条数量为 0 时，只有一种空方案
    all_schemes[0] = [[]]

    # 遍历从 1 到 m 的每个金条数量
    for i in range(1, m + 1):
        # 尝试从之前的方案推导当前方案
        # 情况 1：如果可以从前一个状态转移过来（拿 1 个金条）
        if i - 1 >= 0:
            for prev_scheme in all_schemes[i - 1]:
                new_scheme = prev_scheme + [1]
                all_schemes[i].append(new_scheme)
        # 情况 2：如果可以从前两个状态转移过来（拿 2 个金条）
        if i - 2 >= 0:
            for prev_scheme in all_schemes[i - 2]:
                new_scheme = prev_scheme + [2]
                all_schemes[i].append(new_scheme)

    return all_schemes[m]
```

```python
# 读取输入
m = int(input("请输入金条的数量："))
# 调用函数找出所有方案
schemes = find_all_schemes(m)
print(schemes)
```

根据提供的卡片兑换规则进行兑换计算。已知绿色卡片（green card）1 张需要 10 个代币（token），黄色卡片（yellow card）1 张需要 6 张绿色卡片，白色卡片（white card）1 张需要 2 张黄色卡片。输入拥有的代币数量（以红色字体输入），输出可以得到的绿色卡片、黄色卡片和白色卡片的数量。

```
token = int(input("请输入拥有的代币数量: "))
green_card = token // 10
yellow_card = green_card // 6
white_card = yellow_card // 2
green_card = green_card % 6
yellow_card = yellow_card % 2
print(f"{green_card} green card, {yellow_card} yellow card, {white_card} white card")
```

```c
#include <stdio.h>

int main() {
    int token;
    int green_card, yellow_card, white_card;

    printf("请输入拥有的代币数量: ");
    scanf("%d", &token);

    green_card = token / 10;
    yellow_card = green_card / 6;
    white_card = yellow_card / 2;
    green_card = green_card % 6;
    yellow_card = yellow_card % 2;

    printf("%d green card, %d yellow card, %d white card\n", gr
een_card, yellow_card, white_card);

    return 0;
}
```

实现一个石头剪刀布游戏的结果判定程序。两个玩家分别为 `player1` 和 `player2`，他们各自输入代表石头（R）、剪刀（S）、布（P）的字符。程序根据输入输出游戏结果，结果有三种情况：平局（`Nobody wins`）、玩家 1 获胜（按照“石头砸剪刀（`Rock break Scissors`）”、“剪刀剪布（`Scissors cut Paper`）”、“布包石头（`Paper cover rocks`）”的规则判定）、玩家 2 获胜。

```
player1 = input("Player1: ").upper()
player2 = input("Player2: ").upper()
if player1 == player2:
    print("Nobody wins")
elif (player1 == "R" and player2 == "S"):
    print("Rock break Scissors")
elif (player1 == "S" and player2 == "P"):
    print("Scissors cut Paper")
elif (player1 == "P" and player2 == "R"):
    print("Paper cover rocks")
else:
    if player2 == "R" and player1 == "S":
        print("Rock break Scissors")
    elif player2 == "S" and player1 == "P":
        print("Scissors cut Paper")
    else:
        print("Paper cover rocks")
```

```c
#include <stdio.h>
#include <ctype.h>

int main() {
    char player1, player2;
    printf("Player1: ");
    scanf(" %c", &player1);
    player1 = toupper(player1);
    printf("Player2: ");
    scanf(" %c", &player2);
    player2 = toupper(player2); //tolower()

    if (player1 == player2) {
        printf("Nobody wins\n");
    } else if ((player1 == 'R' && player2 == 'S') ||
               (player1 == 'S' && player2 == 'P') ||
               (player1 == 'P' && player2 == 'R')) {
        printf("Player 1 wins\n");
    } else {
        printf("Player 2 wins\n");
    }

    return 0;
}
```

Implement a result - judging program for the Rock - Paper -
 Scissors game. The two players are player1 and player2 respectively, and they each input a charac ter representing Rock (R), Scissors (S), and Paper (P). The program outputs the game result accordi ng to the input. There are three situations for the result: a draw (Nobody wins), player 1 wins (judg ed according to the rules of "Rock break Scissors", "Scissors cut Paper", "Paper cover rocks"), and p layer 2 wins.

假设你在银行有两种存款方式：单利（simple interest）和复利（compound interest）。编写一个 Python 程序，用户输入本金（principal）、年利率（annual_interest_rate，以小数形式表示，如 0.05 代表 5%）、存款年限（years）以及计息方式（"simple" 表示单利，"compound" 表示复利），程序输出到期后的总金额。

```
principal = float(input("请输入本金: "))
annual_interest_rate = float(input("请输入年利率（小数形式）: "))
years = int(input("请输入存款年限: "))
interest_type = input("请输入计息方式（simple/compound）: ")

if interest_type == "simple":
    total_amount = principal * (1 + annual_interest_rate * years)
elif interest_type == "compound":
    total_amount = principal * (1 + annual_interest_rate) ** years
else:
    print("无效的计息方式")
    total_amount = 0

print(f"到期后的总金额为: {total_amount}")
```

Suppose you have two ways of depositing money in the bank: simple interest and compound i
nterest. Write a Python program where the user inputs the principal, the annual interest rate (e
xpressed as a decimal, e.g., 0.05 represents 5%), the number of years of the deposit, and the ty
pe of interest calculation ("simple" for simple interest, "compound" for compound interest). Th
e program should output the total amount at maturity.

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** *`target`* 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

示例 1：

> **输入：** `nums = [2,7,11,15], target = 9`
> **输出：** `[0,1]`
> **解释：** 因为 `nums[0] + nums[1] == 9` ，返回 `[0, 1]` 。

示例 2：

> **输入：** `nums = [3,2,4], target = 6`
> **输出：** `[1,2]`

示例 3：

> **输入：** `nums = [3,3], target = 6`
> **输出：** `[0,1]`

```python
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]


nums = [2, 7, 11, 15]
target = 9
print(twoSum(nums, target))
```

Perfect number 完美数：完美数是指一个正整数，它的所有因子（除了它本身以外）的和恰好等于它本身。换句话说，如果一个数的因子和等于它自己，那么它就被称为完美数。例如，第一个完美数是 6，因为它的因子包括 1、2、3，而且它们的和也是 6。类似地，28、496、8128 等数也都是完美数。写一个程序，完成以下目标：输入 k，返回第 k 个完美数。例如输入 4，输出“第四个完美数是 8128”

```python
def is_perfect(n):
    sum_factors = 0
    for i in range(1, n):
        if n % i == 0:
            sum_factors += i
    return sum_factors == n


k = int(input("请输入k: "))
count = 0
num = 2
while True:
    if is_perfect(num):
        count += 1
        if count == k:
            print(f"第{k}个完美数是 {num}")
            break
    num += 1
```

```c
#include <stdio.h>

// 判断一个数是否为完美数
int is_perfect(int n) {
    int sum_factors = 0;
    for (int i = 1; i < n; i++) {
        if (n % i == 0) {
            sum_factors += i;
        }
    }
    return sum_factors == n;
}

int main() {
    int k, count = 0, num = 2;
    printf("请输入k: ");
    scanf("%d", &k);

    while (1) {
        if (is_perfect(num)) {
            count++;
            if (count == k) {
                printf("第%d个完美数是 %d\n", k, num);
                break;
```

给定一个整数数组，编写一个程序，找出数组中出现频率最高的数。如果有多个数出现频率相同且都是最高频率，返回其中任意一个即可。

示例：

输入：[1, 2, 2, 3, 3, 3]

输出：3

```
from collections import Counter

def most_frequent(nums):
    count = Counter(nums)
    max_count = max(count.values())
    for num, freq in count.items():
        if freq == max_count:
            return num

nums = [1, 2, 2, 3, 3, 3]
print(most_frequent(nums))
```

```c
#include <stdio.h>

#define MAX_LEN 100

int main() {
    int nums[] = {1, 2, 2, 3, 3, 3};
    int size = sizeof(nums) / sizeof(nums[0]);

    int values[MAX_LEN];
    int counts[MAX_LEN] = {0};
    int uniqueCount = 0;

    for (int i = 0; i < size; i++) {
        int isUnique = 1;
        for (int j = 0; j < uniqueCount; j++) {
            if (nums[i] == values[j]) {
                counts[j]++;
                isUnique = 0;
                break;
            }
        }
        if (isUnique) {
            values[uniqueCount] = nums[i];
            counts[uniqueCount]++;
```

给定一个整数数组，编写一个程序，找出数组中乘积最大的两个整数。数组中可能存在负数。

示例：

输入：[3, -20, 5, -4, 2]

输出：120 （因为 -20 * -4 = 80，3 * 5 = 15，-20 * 2 = -40，而 5 * 2 = 10，最大乘积为 -20 * -4 = 80）

```python
def max_product(nums):
    nums.sort()
    n = len(nums)
    product1 = nums[-1] * nums[-2]
    product2 = nums[0] * nums[1]
    product3 = nums[0] * nums[-1]
    return max(product1, product2, product3)

nums = [3, -20, 5, -4, 2]
print(max_product(nums))
```

```
#include <stdio.h>

// 划分函数，用于快速排序
int partition(int nums[], int low, int high) {
    int pivot = nums[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (nums[j] <= pivot) {
            i++;
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }
    }
    int temp = nums[i + 1];
    nums[i + 1] = nums[high];
    nums[high] = temp;
    return i + 1;
}

// 快速排序函数
void quickSort(int nums[], int low, int high) {
    if (low < high) {
```

```c
#include <stdio.h>

// 交换两个整数的值
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// 冒泡排序
void sort(int nums[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (nums[j] > nums[j + 1]) {
                swap(&nums[j], &nums[j + 1]);
            }
        }
    }
}

int max_product(int nums[], int n) {
    sort(nums, n);
    int product1 = nums[n - 1] * nums[n - 2];
```

给定一个由整数组成的数列，编写一个 Python 程序判断该数列是否为单峰数列。如果是单峰数列则输出 `1`，否则输出 `0`。单峰数列是指一个数列 `a1, a2, ..., an` 满足 `a1 < a2 < ... < ai > ai+1 > ... > an` 的数列。

示例：

输入：`[1, 3, 5, 4, 2]`

输出：`1`

输入：`[1, 2, 3, 4, 5]`

输出：`0`

```python
def is_peak_sequence(nums):
    if len(nums) < 3:
        return 0
    increasing = True
    peak_found = False
    for i in range(1, len(nums)):
        if increasing:
            if nums[i] < nums[i - 1]:
                increasing = False
                peak_found = True
        else:
            if nums[i] >= nums[i - 1]:
                return 0
    return 1 if peak_found else 0

nums = list(map(int, input("请输入一个整数数列，以空格分隔：").split()))
print(is_peak_sequence(nums))
```

有 12 堆硬币，每堆有 12 枚硬币。其中恰好有一堆硬币全是假币（即伪造的），其余 11 堆的硬币都是真币（不是假的）。每枚真币重 10 克，每枚假币重 11 克。你有一个可以告诉你任意所选硬币总重量的秤。设计一种使用该秤找出假币堆的方法（一种明显的解决方案是使用秤 12 次，寻找一种使用秤次数少于 12 次的方法）。

```
# 假设第fake_stack堆是假币堆，这里只是为了模拟，实际开始时并不知道
fake_stack = 5
# 初始化12堆硬币，每堆12枚
stacks = [[10] * 12 for _ in range(12)]
# 将假币堆的硬币重量设为11克
stacks[fake_stack][:] = [11] * 12

# 模拟称重找假币堆的过程
total_weight = 0
for i in range(12):
    total_weight += (i + 1) * stacks[i][0]

# 计算理论上全是真币时的重量
expected_weight = sum(range(1, 13)) * 10
# 计算重量差值
weight_difference = total_weight - expected_weight
# 根据差值确定假币堆
if weight_difference != 0:
    result_stack = weight_difference - 1
    print(f"假币堆是第 {result_stack + 1} 堆")
else:
    print("未找到假币堆，模拟数据可能有误")
```

假设 `is_prime(m)` 是一个内置函数，如果 `m` 是质数，它返回 `True`，否则返回 `False`。

一个正整数是特殊的，当且仅当它有两个因数 `f` 和 `f + 2`，并且 `f` 和 `f + 2` 都是质数（即 `f` 和 `f + 2` 构成一对孪生质数）。

编写一个完整的程序（使用任何编程语言），输入一个正整数 `x`，如果 `x` 是特殊的，打印 “Yes”，否则打印 “No”。你可以在程序中使用 `is_prime()` 函数。（注意：1 是任何正整数的因数。）

```
def is_prime(m):
    if m < 2:
        return False
    for i in range(2, int(m ** 0.5) + 1):
        if m % i == 0:
            return False
    return True


x = int(input("请输入一个正整数: "))
is_special = False
for f in range(1, x + 1):
    if x % f == 0 and is_prime(f) and is_prime(f + 2) and (f * (f + 2) == x or (x // f) * ((x // f) + 2) == x):
        is_special = True
        break
print("Yes" if is_special else "No")
```